BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Artificial Intelligence

ENCS3340

Project 2

Prepared by:

Mohammad Omar **1221332**

Heba Mustafa **1221916**

Instructor: Dr. Yazan Abu Farha

Section: 1

Date: **17/6/2025**

Table of Content

Table of Content I
1. Introduction 1
2. Dataset Description 2
3. Model Descriptions 3
3.1 Naive Bayes Classifier 3
3.2 Decision Tree Classifier 3
3.3 Feedforward Neural Network (MLP Classifier) 4
4. Evaluation Criteria 4
6. Results 5
7. Comparative Analysis 7
8. Extra Implementations 8

1. Introduction

This project compares three machine learning methods: Naive Bayes, Decision Tree, and a Feedforward Neural Network (MLP) on image classification. A balanced pool of more than 500 pictures, evenly spread across three classes, was resized to 64 by 64 pixels and turned into one-dimensional vectors before training. The aim is to judge each technique by how well it labels images, how easy its decisions are to explain, and the time and memory it consumes. Naive Bayes, praised for its speed and straightforward math, sets the first benchmark. The tree, with its human-readable branches, offers clear rules that analysts can follow. The MLP digs deeper into the pixel data, learning complex, non-linear patterns that the other two might miss. Performance is measured through accuracy, precision, recall, F1-score, and confusion matrices so readers can see failures as well as successes. Results confirm the familiar trade-off; the MLP usually wins on raw accuracy, yet the lighter models still matter because they run faster and tell clear stories. These findings help teams choose the right tool for classifying images when speed, insight, or sheer prediction power is the most urgent concern.

U can use our website from this link https://mlclassification.streamlit.app/

- Username: admin
- Password: 1234

2. Dataset Description

The dataset consists of color images collected from various online sources such as Kaggle, arranged into three distinct fruit categories: **blueberries, pomegranates, and bananas**. To ensure balance and fairness in model comparisons, we selected an equal number of images from each category: 513 images per category (1539 images in total).

1. Image size: Each image was resized to 64 x 64 pixels.
2. Color channels: RGB (3 channels)
3. Preprocessing: Images were resized and converted to RGB.
4. The resulting feature vectors were flattened for input into traditional classifiers

We perform two operations on the images:

1. Background Removal: AI-based segmentation (rembg) cuts the fruits from backgrounds to become RGBA images (transparent background).
2. Train/Test Split: Stratified 5-fold cross-validation (20% test size).
3. Ensures balanced class representation in splits.

Figure 1: Dataset Categories

3. Model Descriptions

3.1 Naive Bayes Classifier

Naive Bayes operates on probabilities, based on Bayes' rule. It treats each feature as if it were working alone, so it works quickly on large inputs such as image networks. For this task, we chose scikit-learns Gaussian NB because our data is often connected. We implement the Naïve Bays as the following: -

1. Feature Extraction:

   Statistical Features are Grayscale mean & std and Mean & std for each R, G, B channel which give us 6 features.

2. Training: Gaussian distribution estimation for each feature and standardization for variance normalization.
3. Prediction: We calculate posterior probabilities for each class.

- Pros: Very fast, requires little memory.
- Cons: The independence story rarely matches the actual pixels.

3.2 Decision Tree Classifier

A decision tree travels a yes/no path through the data, starting from the root.. We implement the Decision Tree as the following using the scikit learn model: -

1. Feature Extraction:

Divide image into 4x4 blocks (16 blocks total). And for each block, we compute mean R, G, B values which gave us 46 features.

2. Training: we use the following pre-pruning parameters (the user can change them from the Configuration Panel)
   max_depth=10, min_samples_split=10, min_samples_leaf=5, min_impurity_decrease=0.01.
   Too minimize the complexity and remove unnecessary splits we used post pruning using ccp-alpha (cost complexity alpha) = 0.001
   These are all parameters that with proper tuning minimize overfitting and also help the overall performance

3. Prediction: Traverse tree nodes using RGB block features.

- Pros: Easy to read, can be plotted.
- Cons: Preserves noise if the tree gets too deep.

3.3 Feedforward Neural Network (MLP Classifier)

MLP, or Multi-Layer Perceptron, is a simple feedforward network built using the MLP Classifier sklearns. It hides one or more layers between the input and output. We tested one hidden layer with 128 neurons. We implement the module as the following: -

1. Feature Extraction:

   Transfer Learning: we use Pre-trained VGG16 (ImageNet weights).

   Output: 2x2x512 feature maps to 2048 features.

2. Classifier: we build a MLP with one hidden layer that contain 128 neurons and the output layer with 3 neurons.
3. Training: All inputs are CNN vectors scaled using Standard Scaler.

- Advantages: Learns sharp and curved decision boundaries.
- Cons: takes longer to train.

Note: for additional information about System Architecture Overview u can find it in the bottom of the webpage.

4. Evaluation Criteria

We evaluated every model with these straightforward measures:

1. Accuracy: Share of correct guesses out of all made.
2. Precision: Hits divided by hits plus the false alarms.
3. Recall: Hits stated divided by hits missed.
4. F1-score: The tidy average of precision and recall.
5. Confusion matrix: A table that counts each class correct and wrong calls.

Note: All tests ran on a 20 percent hold-out split of the balanced data.

6. Results

We try a sample run five times then we take the average between them and the output as the following: -

Final Results Summary

DECISION TREE	NAIVE BAYES	CNN + MLP
90.58%	87.21%	98.31%

Naive Bayes

Class	Precision	Recall	F1-Score	Support
banana	0.874	0.796	0.833	514
blueberries	0.914	0.971	0.942	515
pomegranates	0.827	0.849	0.838	511
accuracy	0.872	0.872	0.872	1
macro avg	0.872	0.872	0.871	1,540
weighted avg	0.872	0.872	0.871	1,540

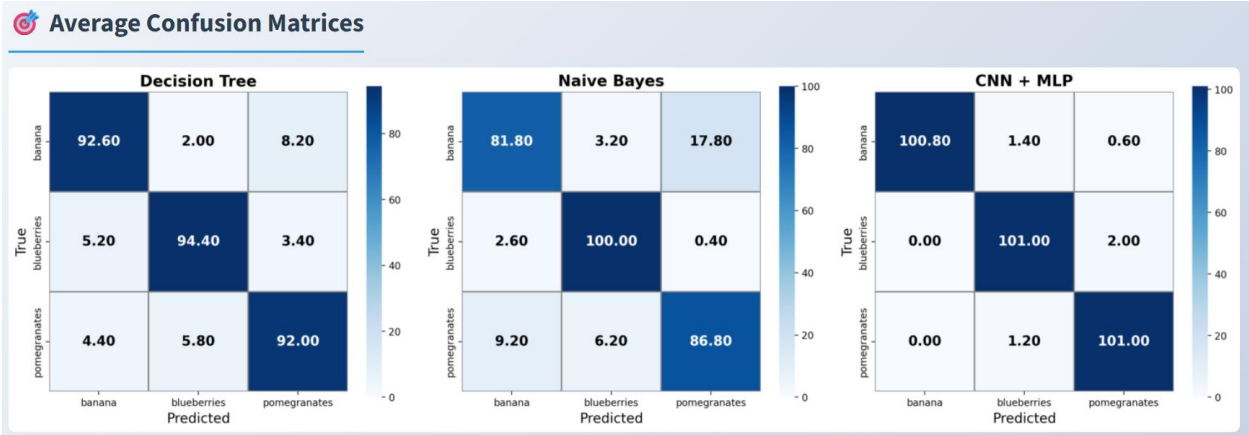
Classification Reports

Decision Tree

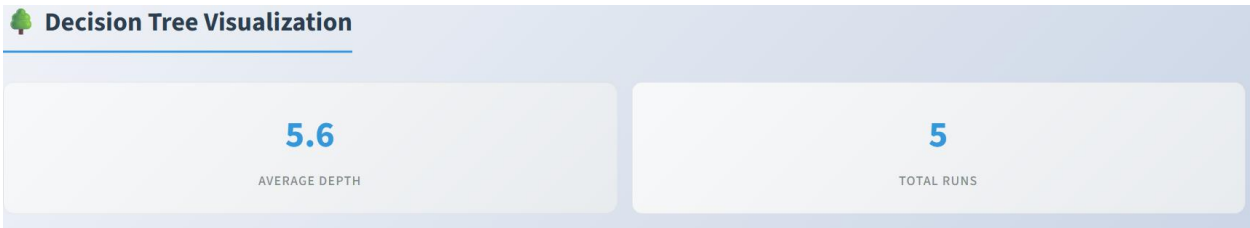
Class	Precision	Recall	F1-Score	Support
banana	0.906	0.901	0.903	514
blueberries	0.924	0.917	0.920	515
pomegranates	0.888	0.900	0.894	511
accuracy	0.906	0.906	0.906	1
macro avg	0.906	0.906	0.906	1,540
weighted avg	0.906	0.906	0.906	1,540

CNN + MLP

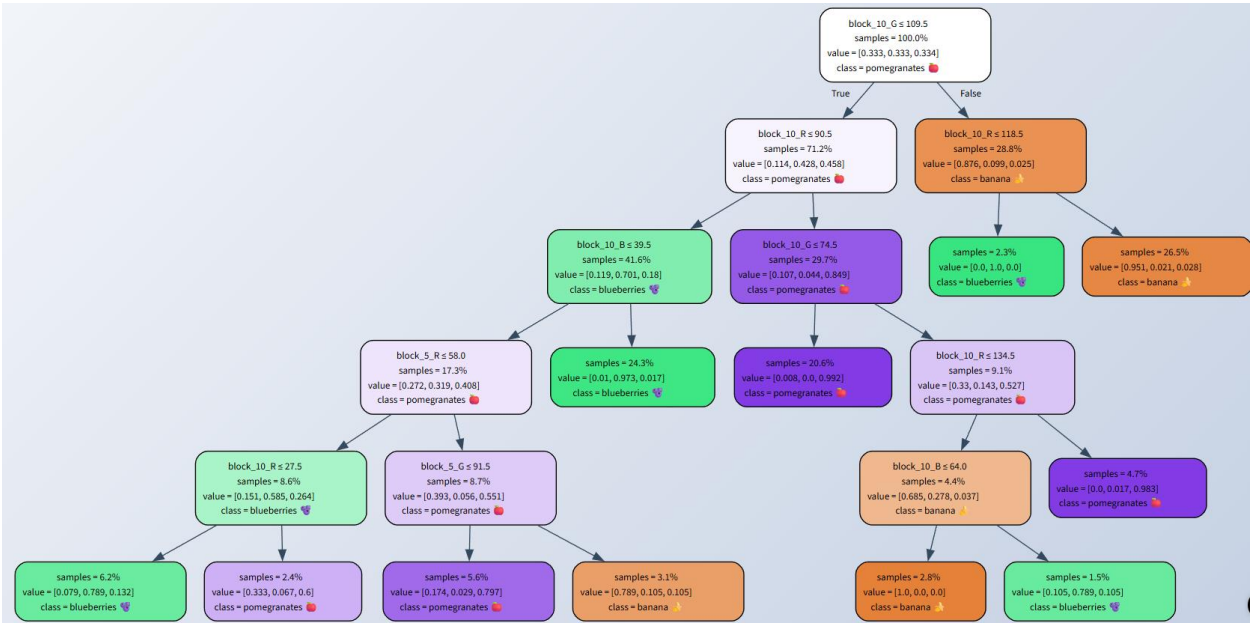
Class	Precision	Recall	F1-Score	Support
banana	1.000	0.981	0.990	514
blueberries	0.975	0.981	0.978	515
pomegranates	0.975	0.988	0.982	511
accuracy	0.983	0.983	0.983	1
macro avg	0.983	0.983	0.983	1,540
weighted avg	0.983	0.983	0.983	1,540



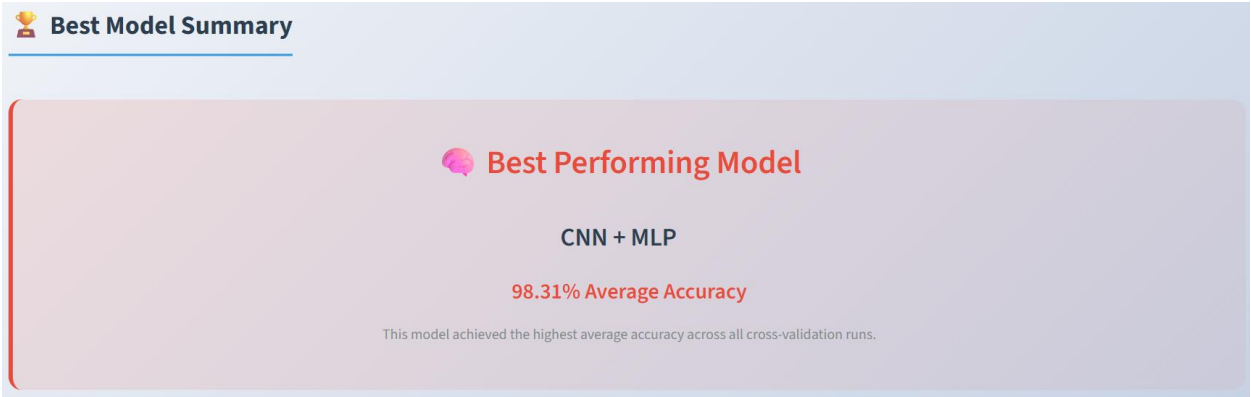
For the decision trees: -



We take a screenshot of one of the five runs as the following



Here we can find that the best performing model is the CNN + MLP (neural network).



7. Comparative Analysis

The naive Bayes algorithm was fast and simple, but its performance was poor due to its assumption of feature independence.

The decision tree improved this performance, but showed signs of overfitting, especially when depth was not constrained.

MLP Classifier clearly outperformed both, leveraging its nonlinear transformation capabilities and rich features in convolutional neural networks (CNNs). However, its training time was longer.

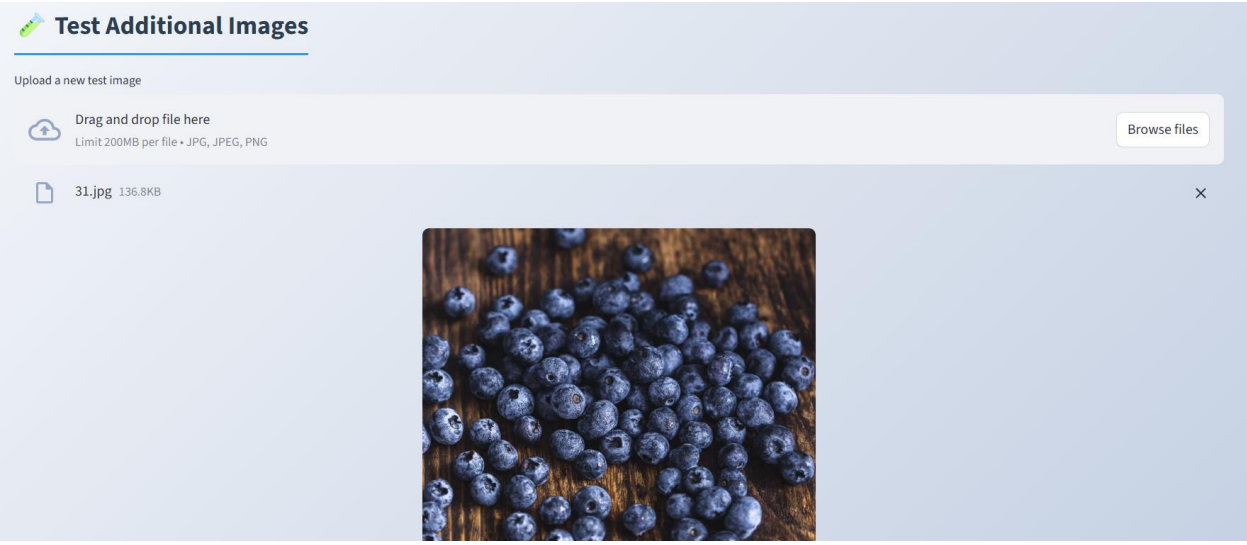
We can summarize the Comparative analysis in the following table: -

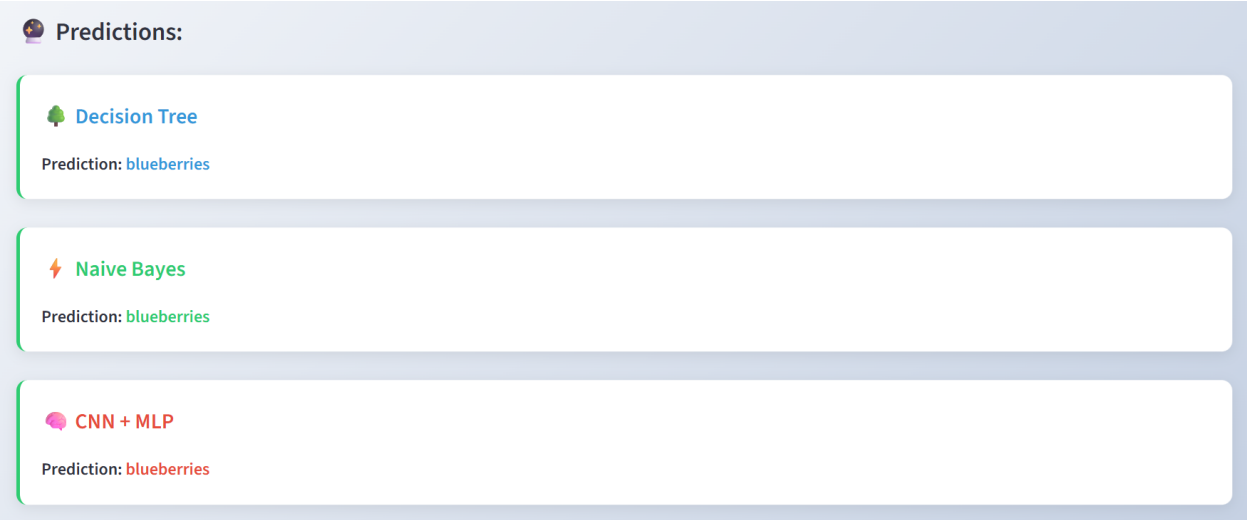
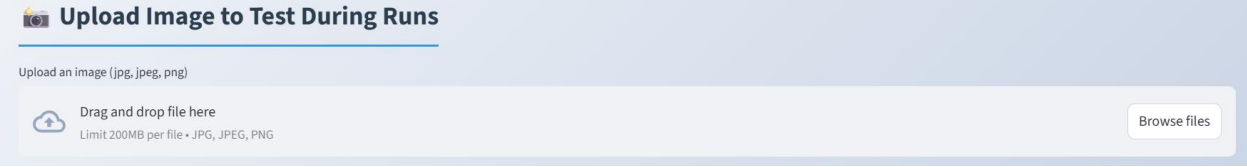
Table 1: Comparative analysis

Model	Training Time	Strengths	Weaknesses
Naive Bayes	Very Fast	Simple, fast, low memory usage	Poor performance due to feature independence
Decision Tree	Fast	Interpretable, captures patterns	Prone to overfitting
MLP (1 hidden layer)	long	Captures nonlinear features well	Longer training time

8. Extra Implementations

We give freedom to the user that he can upload image and then it will be classified according to the final avg run for the program like the following: -





<u>Note</u>: user can run the image classification based on the average result (in the bottom of the web page) and he can do the process in the hole times of runs.